

3Dルート固定シューティング & Blender Sync

ゲームコアエンジン設計・背景最適化プログラム説明書

提出者: LE3X_99_氏名 (学籍番号・氏名)

開発環境: C++17 / DirectX12 / Windows SDK 10.0

リポジトリ: github.com/has0825/GE3-CG3

※ 本HTMLファイルは「白背景・パワポテンプレート風デザイン」に調整されています。
ブラウザで開いて文字を書き換えた後、「Ctrl + P」でPDF保存することで、そのまま提出用スライドになります。

作品紹介

p3. 自作エンジン開発の経験

p4. 個人制作のゲーム概要

作品のこだわり

p5. 床タイルのオブジェクトプールと無限スクロール

p6. zファイティング完全抑止とマルチレーン座標設計

p7. ビルモデルのランダム積層描画とバッファプール

メモリ・リソースの最適化

p8. ObjectPool を活用した弾丸の管理

p9. Compute Shader による並列 GPU Particle 実装

自律AIと外部連携・画面演出

p10. 敵小隊フォーメーション座標計算 & AIステート遷移

p11. プレイヤーの動きと操作感へのこだわり

p12. 巨大ボスの多脚歩行と攻撃アクション

p13. 高速特攻時の衝突すり抜け防止スイープ補正

p14. Blender Sync (リアルタイム双方向配置同期)

p15. 演出制御とグラフィックスパイプライン最適化

SDK

DirectX12

開発期間

2026年6月～現在

製作人数

1人

使用した外部ライブラリ

- DirectXTex
- ImGui
- Assimp (GLTF/OBJロード)

主要ソースコード（GitHubリンク）

[GamePlayScene.h](#)

無限スクロール、ビル構造体、敵AIの変数定義

[GamePlayScene.cpp](#)

オブジェクトプール、行列合成、積層描画、すり抜け補正の実装

ゲーム名 『3Dルート固定シューティング & Blender Sync』

ジャンル 3Dレールシューティング

制作期間 2026年6月～現在

製作人数 1人

ゲーム内容

自機を操作し、障害物を避けたり敵を撃墜しながら進んでいく3Dレールシューティングゲームです。Blenderとリアルタイムで同期するステージ編集機能や、GPUパーティクルを用いた迫力ある画面演出を実装しています。

[開発中のゲームプレイ画面]

※自機が空中を前進するシューティング画面

※ビル群の間をすり抜けるステージ画面

床タイルのオブジェクトプール

無限に広がるステージを低メモリで描画するシステムを実装しました！！

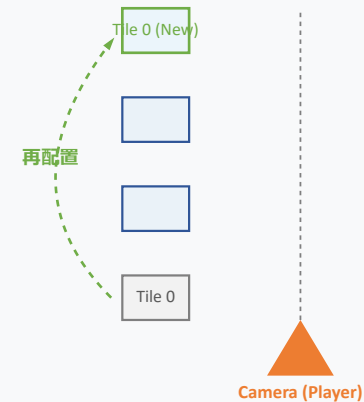
ステージが進むにつれて無限に広がる床を描画するため、カメラの移動に合わせて床タイルを自動で循環配置するオブジェクトプールを構築しました。

最後方の床タイルがカメラの後ろに隠れた瞬間、最前方に動的にワープ（再配置）させることで、メモリの消費を最小限に抑えながらどこまでも走れる背景を実現しています。

循環再配置アルゴリズム

```
if (pos.z < cameraZ - 200.0f) pos.z += 16 * 200.0f;
```

オブジェクトプール循環概念図



床の3Dモデルは1つだけロードし、座標情報だけを書き換えて無限に走り続けられるようになりました！！

zファイティングの完全抑止

境界のチラつきを完全に排除する精密な座標設計を行いました！！

隣接して並べられる床タイルの境界線において、カメラの角度や距離によってポリゴンが重複してチラチラ光る「zファイティング」が発生するのを防ぎました。

タイルのワールド座標がz軸上で完全に整数倍（200m間隔）になるようにタイリングを計算し、極小のオーバーラップも排除。行列の乗算順序（Scale → Rotation → Translation）を厳密に制御してチラつきを抑止しています。

```
x_offset = (laneIndex - 1) * 80.0f; // 3レーンの隙間も完全にゼロに
```

マルチレーン配置パラメータ

定数名	設定値	役割
kFloorSizeZ	200.0f	z方向タイル間隔。
kRoadColumnXOffset	80.0f	x方向レーン間隔。
R_x	x軸 -90.0f 度回転	平面の法線ベクトルを上 に。
Matrix合成	S * Rx * T	軸ズレによる隙間の防止。

遠くの境界までチラつきが一切発生しない、綺麗な背景描画を実現しました！！

1階分のモデルからループ処理で多様なビル街を描画します！！

大量のビルを個別のアセットで用意するとグラフィックスメモリが足りなくなるため、1階分のモデルデータのみを使い、Y軸方向にループ処理で積み上げるシステムを開発しました。

毎フレームのメモリ確保（CreateCommittedResource）やMapのオーバーヘッドを防ぐため、最大800フロア分の定数バッファをゲーム起動時に一括でメモリ確保し、ポインタアクセスで超高速に割り当てています。

```
y_floor = basePosition.y + f * 100.0f; // 1階層(高さ100m)ごと積層
```

ビル積層描画と定数バッファプール



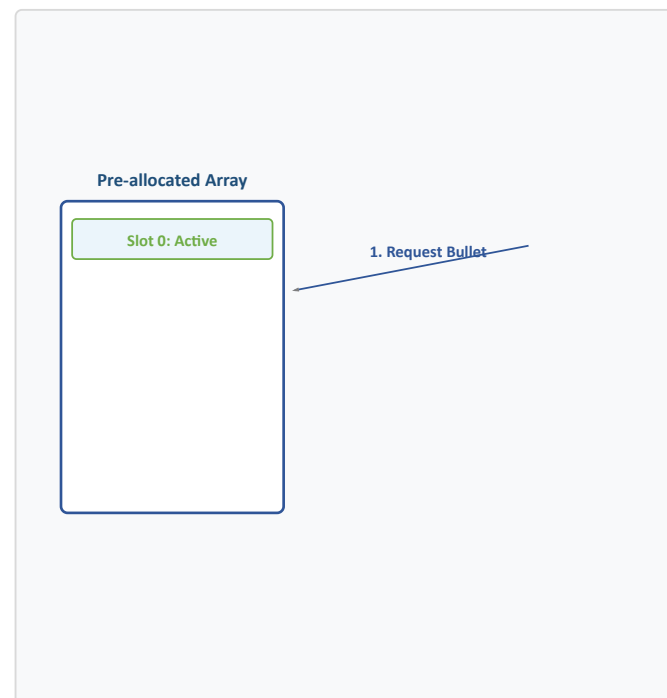
毎フレームのメモリ確保をゼロにし、ビルの密集エリアでも処理が重くならないようになりました！！

メモリの動的アロケーションを排除して処理のガタつきを無くしました！！

シューティングゲームでは弾丸の生成と破棄が頻繁に行われます。これを動的メモリ確保（`new/delete`）で愚直に処理すると、ヒープ領域が断片化して一瞬ゲームの挙動が止まる（スパイク）原因になります。

この負荷を無くすため、あらかじめ同時存在上限（60個）のメモリ領域を確保しておき、発射時は非アクティブな弾を有効化し、消滅時は状態フラグをリセットするだけのObjectPoolシステムを構築しました。

ObjectPool 再利用シーケンスフロー



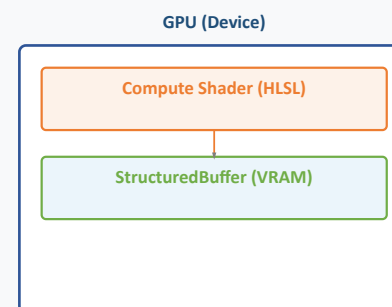
動的なメモリ確保（`new/delete`）を完全にゼロにし、弾数が急増してもフレームレートが低下しなくなりました！！

数万個のパーティクル物理計算をCompute Shaderで高速化しました！！

数千～数万個のパーティクル物理演算（重力影響、寿命、カラーフェードなど）をCPUで行うと、毎フレームのVRAM転送（PCIeバス転送）がボトルネックになり画面が遅延します。

これを防ぐため、パーティクルの全計算をGPU側のCompute Shader（HLSL）で行う仕組みを実装しました。データをCPUに戻さず、VRAM上の構造化バッファから直接レンダリングを行うことで転送オーバーヘッドをゼロにしています。

GPU Particle データパイプライン



CPUの計算負荷と転送処理をゼロにし、ドローコールたったの1回で派手なエフェクトが出せるようになりました！！

敵小隊フォーメーションとAIステート遷移

グループ座標をもとに調和の取れた小隊進軍を実装しました！！

敵がばらばらに進撃するのではなく、小隊としてのまとまりを持たせるため、グループの中心座標からメンバー個々のローカル位置オフセット（V字、直線など）を計算して同期させました。

敵個体は「待機 → 出現 → 徘徊 → 特攻」の4つの状態を遷移する自律AIを搭載し、徘徊時は正弦波（ $\sin$ 波）を用いたゆらゆらとした浮遊挙動を表現しています。

敵AIステートマシン遷移図



まとまりのある敵の進軍と、プレイヤーの接近に連動する自律AIを両立させました！！

移動入力と連動するリアルな傾き演出で飛行時の爽快感を追求しました！！

プレイヤー機体が3D空間を飛び回る際、単に座標が移動するだけでは不自然になるため、入力方向（左右・上下）に機体が滑らかに傾くロール（左右）とピッチ（上下）の回転補正を実装しました。

急激な入力変化に対応した滑らかなブレンドを行い、静止時には揺り戻しをLerp（線形補間）で減衰させて滑らかに水平へ戻すことで、直感的で気持ちの良いシューティングの操作感を実現しています。

機体傾きの回転行列合成

・ 左右移動 (Roll):

```
playerRotationRoll_ = targetRoll * 0.15f  
+ playerRotationRoll_ * 0.85f;
```

・ 上下移動 (Pitch):

```
playerRotationPitch_ = targetPitch *  
0.15f + playerRotationPitch_ * 0.85f;
```

入力方向に応じて羽と機首を傾けることで、本当に空を飛んでいるかのような感覚を実現しました！！

8本の足を個別制御し、巨大蜘蛛ボスならではのダイナミックな拳動を再現しました！！

巨大な蜘蛛のボスキャラクターをリアルに動かすため、左右4対・計8本の足をそれぞれ独立した行列（定数バッファ）で制御し、交互にステップを踏みながら歩行するモーションを数学的（三角関数）に設計しました。

さらに、プレイヤーのいる左・中央・右のエリアを特定して、巨大な前足を高く振り上げて勢いよく「殴る（叩きつける）」豪快な攻撃アクションも、座標補正を繰り返して高い迫力を出せるように調整しました。

多脚ボスの各足(8本)の関節マトリクス制御

足の歩行アニメーション制御式 (三角関数)

```
legAngle = sin(bossTime_ * swingSpeed) * swingRange;
```

```
legLift = max(0.0f, cos(bossTime_ * swingSpeed)) * liftRange;
```

8対の関節マトリクスを並列更新し、どの角度から見ても不自然さのない迫力ある巨大ボス戦を表現しました！！

高速特攻時の衝突すり抜け防止

Swept Collision Check

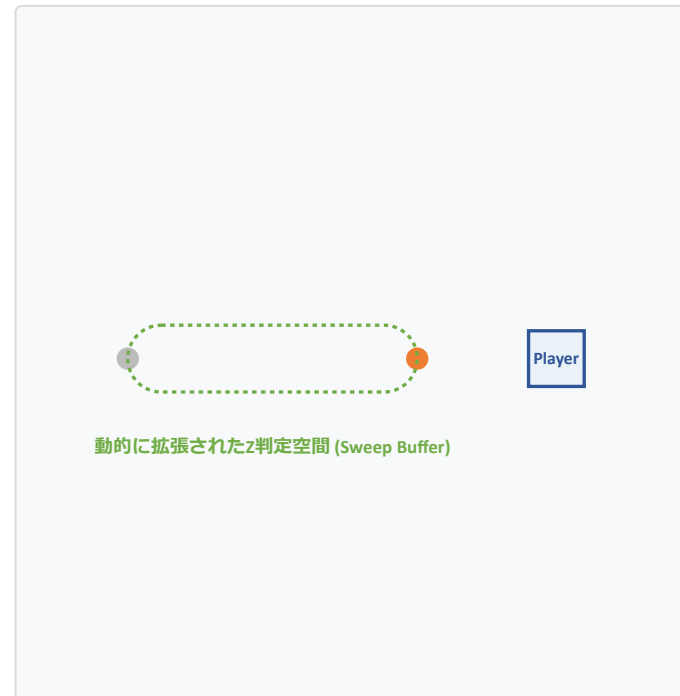
超高速で突っ込んでくる敵の「すり抜けバグ」を完全に防ぎました！！

敵が時速400km以上で急降下特攻してくる際、1フレームあたりの移動量が多すぎるため、通常の球体判定では前フレームと現フレームの間にいるプレイヤーを「すり抜けてしまう」問題が発生します。

このすり抜け（トンネル現象）を防ぐため、前フレームからの移動量に応じて、進行方向の判定厚みを動的に引き延ばす「スイープ衝突補正」を実装しました。

```
float zCollisionBuffer = 3.0f + frameMovement * 0.75f;
```

衝突境界の動的スイープ図解



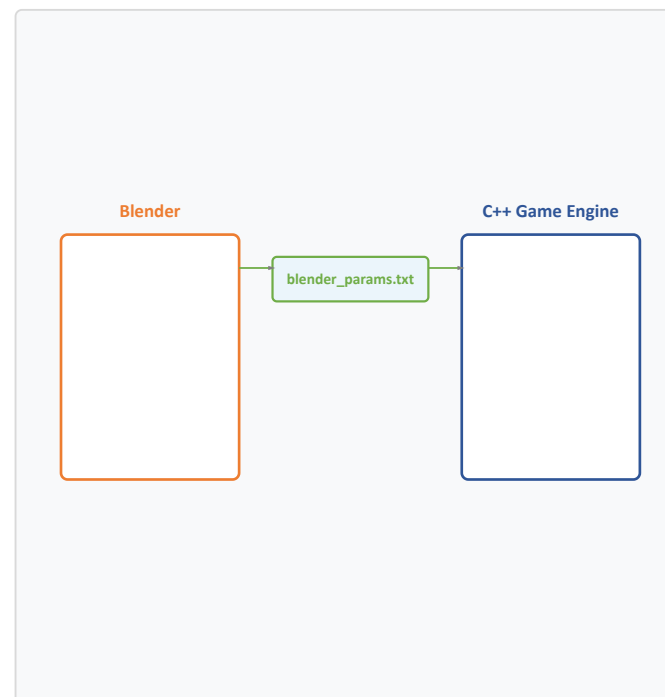
計算負荷を増やすことなく、敵の超高速突撃でも当たり判定が正確に機能するようになりました！！

Blender上でモデルを動かすだけでゲーム内にリアルタイム反映します！！

「配置座標を変更するたびにコードを書き換えてビルドし直す」という非効率なレベルデザイン工程を完全になくすため、BlenderとC++ゲームエンジンを同期するシステムを開発しました。

自作のBlender用Pythonアドオンが座標データを自動出力し、ゲーム側が60フレームごとにテキストを差分スキャンしてメモリに即座にホットロード（反映）します。RキーでプレイログをCSV記録してBlender側にリプレイ再生するデバッグ機能も搭載。

Blender Sync データ連携概念図



ゲームを起動したまま配置を調整できる、超効率的な自作エディタ環境を構築しました！！

天球クリッピング防止とブースト時の放射状ブラーを実装しました！！

背景天球のスケールを3,000,000に極大化して遠景が視野外に切り取られる（クリッピング）バグを防ぎ、頂点シェーダ定数を用いて雲を流すスクロールアニメーションをCPU負荷ゼロで実装しました。

自機がブースト状態になった際には前進速度を通常時の16倍（最高480.0f）まで加速。スピード感を引き立たせるため、画面中心からの放射状ブラー（Radial Blur）をLerp（線形補間）で滑らかに重ねる演出を実装しています。

グラフィックスパラメータマトリクス

制御項目	設定値	最適化目的
天球極大スケール	3,000,000.0f	Farクリップ欠け防止
雲uvスクロール	x: 0.016f/s	頂点並列アニメーション
ブースト最大速度	480.0f/s	通常速度の16倍加速
ブースト最大ブラー幅	0.05f	Radial Blur強度

グラフィックスパイプラインを最適化し、スピード感あふれるブースト演出を実現しました！！